

# Hierarchical Multi-Agent Control by LLM: Level-2 Swarm Coordination via MAPF, Formation Control, and Behavior Trees

Evgenii Shlomov  
Innopolis University  
Innopolis, Russia  
e.shlomov@innopolis.university

Anastasia Malakhova  
Innopolis University  
Innopolis, Russia  
a.malakhova@innopolis.university

Senya Belyh  
Innopolis University  
Innopolis, Russia  
s.belykh@innopolis.university

Marat Shariev  
Innopolis University  
Innopolis, Russia  
m.shariev@innopolis.university

**Abstract**—We present the Level-2 component of a three-level hierarchical multi-agent system for natural-language control of a 20-robot differential-drive swarm. Level 2 is responsible for the actual coordination of the fleet—turning fleet-level goals into conflict-free, time-indexed paths and into stable leader–follower formations—while exposing a compact set of atomic primitives to the higher-level reasoning layer. Our implementation, `iros_llm_swarm`, is built on ROS 2 Humble and combines two interchangeable Multi-Agent Path Finding (MAPF) backends—Priority-Based Search (PBS) and Large Neighborhood Search (LNS2)—selectable per task behind a single fleet-level action, a centralized formation manager with a state-machine monitor, per-robot dual-mode followers that switch between Nav2 path-following and a PD offset-tracking controller without restarting, a custom self-clearing costmap layer to remove ghost-obstacle trails, and a set of BehaviorTree.CPP action nodes that wrap every primitive behind a single long-lived action interface. We describe the system design, justify the engineering decisions made under DDS discovery overhead at 20-robot scale (CycloneDDS profile, disabled AMCL, staggered launch), and report evaluation results covering planner correctness, a six-step integration scenario, and robustness observations.

**Index Terms**—multi-robot systems, multi-agent path finding, PBS, LNS2, formation control, behavior trees, ROS 2, Nav2

## I. INTRODUCTION

Coordinating a fleet of mobile robots that share a single workspace is a long-standing problem in multi-robot systems. The classical decomposition separates fleet-level decisions—which robot should go *where* and *in what order*—from robot-level execution—*how* a single robot follows an assigned path or maintains an offset inside a formation. Recent interest in using large language models (LLMs) as high-level decision makers [25]–[28] makes this decomposition even more attractive: the LLM can reason over compact, semantic primitives instead of producing low-level control commands, and the underlying classical stack remains responsible for safety, conflict resolution, and real-time tracking.

The broader project follows a three-level hierarchy: a Level-1 LLM agent translates free-form natural-language commands

into structured tasks; a Level-2 swarm controller executes those tasks using classical multi-agent algorithms; and Level-3 robots, embedded in a simulated environment, execute the resulting motion commands. *This report describes the Level-2 contribution.* The Level-1 agent and the Level-3 simulation/Nav2 spawning are documented separately by the partner team.

## Scope and contributions

We focus on a fleet of 20 differential-drive robots operating in a 2D warehouse-style environment. The Level-2 layer provides:

- conflict-free, time-indexed paths for every robot from arbitrary start poses to a set of assigned goals;
- stable leader–follower formations with online activation, deactivation, and reshaping;
- a single long-lived action endpoint that the higher-level reasoning layer can treat as one atomic operation;
- robust per-robot motion execution that absorbs schedule slip, costmap noise, and Nav2 controller failures.

Our specific contributions are: (1) two interchangeable MAPF backends—PBS with 8-connected A\*, gradient inflation, and capsule-based conflict detection; and LNS2 with adaptive large-neighborhood search over a soft-constraint A\* with warm-start on replanning—sharing the same fleet-level action surface so that the backend can be selected per task without touching the upstream layers; (2) a long-lived `/swarm/set_goals` action that encapsulates the full *validate* → *plan* → *execute* → *replan* lifecycle behind a single goal token; (3) a centralized formation layer with CRUD-style services, a latched configuration topic, and a state-machine monitor; (4) per-robot dual-mode followers that switch between autonomous Nav2 tracking and PD-controlled offset tracking via a single latched message, without node restarts; (5) a `ResettingObstacleLayer` Nav2 costmap plugin that eliminates ghost-obstacle trails caused by laser

cells that fall *between* rays and are never explicitly cleared; and (6) a small set of BehaviorTree.CPP action nodes that wrap every primitive behind a uniform interface, ready for composition by higher-level mission trees.

## II. RELATED WORK

### A. Multi-Agent Path Finding

MAPF has a rich algorithmic literature [1]. Conflict-Based Search (CBS) [2] is the canonical optimal solver, organizing the search as a two-level tree where conflicts are resolved by adding constraints to a high-level node and replanning individual paths at the low level. Priority-Based Search (PBS) [3] relaxes optimality for scalability: a strict priority ordering on agents replaces explicit time-indexed constraints, and each agent plans deterministically against the paths of higher-priority agents. PBS is fast, reproducible, and produces a hard, capsule-style separation between agents. Large Neighborhood Search for MAPF (LNS2) [4] is an anytime, suboptimal solver that begins with any feasible plan—possibly with collisions—and iteratively destroys and repairs subsets of agent paths using soft-constraint A\* with collision penalties rather than hard blocks; it scales to dense fleets and benefits from warm-starting on replanning. We adopt both PBS and LNS2 as first-class backends behind a shared action interface, and select between them per task: PBS for sparser scenarios where determinism and schedule predictability matter; LNS2 for dense and cluttered scenarios where the anytime budget and soft constraints are decisive.

### B. Formation control

Formation control of multi-robot systems is typically posed in one of three frameworks: leader–follower, virtual structure, or behavior-based. Surveys [7], [9] compare model-based and learning-based methods. Recent work fuses APF with sliding-mode estimators [6] for cooperative formation control, and particle swarm optimization has been applied to differential drive motion planning [8]. Spatial segregative behaviors using differential potentials have also been demonstrated in robotic swarms [5]. Our Level-2 layer uses the leader–follower variant because it interacts cleanly with a per-robot motion controller and is the most natural fit for the kind of semantic offsets a higher-level agent is likely to specify.

### C. Robot navigation stacks

Nav2 [12], [13] is the de facto ROS 2 navigation stack. Layered costmaps [15] are the standard representation for context-sensitive obstacle handling, with raytracing used to clear cells along a sensor ray. Our `ResettingObstacleLayer` addresses a gap in this scheme: cells that fall *between* discrete laser rays are never explicitly cleared and accumulate as ghost obstacles after dynamic objects move.

### D. Behavior trees

Behavior trees are a modular control architecture for robotic systems [16], [17]. BehaviorTree.CPP [18] is the implementation we use; it integrates with ROS 2 actions and provides

a standard XML schema. Behavior trees are particularly suitable when an external reasoning component supplies the tree structure or its parameters.

### E. ROS 2 and DDS at scale

ROS 2 [14] is built on top of the Data Distribution Service (DDS) middleware. At scale, DDS discovery overhead becomes the dominant cost [19], [20], and tuning the RMW implementation is required to keep multi-robot systems responsive. We use CycloneDDS with a project-specific profile that limits unicast discovery and tunes participant settings for 20 simultaneous robots.

### F. LLM-driven robotics

SayCan [25], Code as Policies [26], ProgPrompt [28], and Inner Monologue [27] motivate the architectural choice to expose Level-2 capabilities as a small, well-typed primitive surface. The behavior-tree front-end in our stack is shaped by these patterns.

### G. Multi-robot simulation

Stage [21] and Gazebo [22] are the standard 2D and 3D simulators in the ROS ecosystem. Stage was selected by the partner team for this project; see their report for the Gazebo–Stage trade-off.

## III. METHODOLOGY

### A. System architecture

The Level-2 stack is organized as a vertical pipeline. The simulator publishes ground-truth state (`/tf`, `/robot_*/odom`, `/robot_*/scan`). The active MAPF planner—either PBS or LNS2, selected at launch time—consumes the static map and the per-robot odometry, and produces time-indexed plans on `/robot_*/mapf_path` (PBS) or `/robot_*/mapf_plan` (LNS2). Each robot runs one dual-mode follower paired with the active planner: depending on the latched `/formations/config` signal, the follower either dispatches the plan to Nav2 via `/robot_*/follow_path` or runs a PD controller that publishes directly to `/robot_*/cmd_vel`. The behavior tree sits on top and exposes a single mission action.

### B. Shared action interface and backend selection

Both planners own the same `/swarm/set_goals` action server (only one is active in a given launch). The action contract—goal type, feedback type, lifecycle, cancellation semantics—is identical regardless of which planner is running. Backend selection is a launch-time decision: `swarm_mapf.launch.py` brings up the PBS stack, `swarm_lns.launch.py` brings up the LNS2 stack, and the `*_formation.launch.py` variants add the formation layer to either choice.

PBS is preferred when the scenario is sparse, when schedule predictability matters, and when the clean capsule-style separation between agents is an asset. LNS2 is preferred for dense or cluttered scenarios, for missions that will trigger many replans (its warm-start makes amortized cost low), and whenever transiently infeasible start configurations are likely.

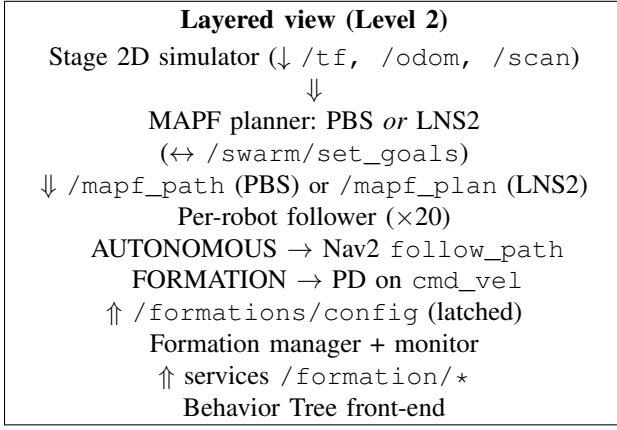


Fig. 1. Level-2 layered architecture. Two MAPF backends share the same `/swarm/set_goals` action; the behavior tree exposes that endpoint and the formation services to the upstream reasoning layer.

### C. MAPF planners

1) *PBS planner*: The PBS implementation, `mapf_planner_node` in `iros_llm_swarm_mapf`, is a C++17 ROS 2 node. The low-level path planner is an 8-connected A\* with a backward-Dijkstra heuristic precomputed from each agent’s goal on the static map. Inflation is treated as a two-zone gradient: a hard inner footprint corresponding to the robot body, and a soft outer penalty zone whose cost decays smoothly with distance, so that A\* prefers wider clearances when available without forbidding tight passages. Conflict detection is capsule-based: each agent’s swept volume is represented as a sequence of capsules in the space-time grid, and pairwise intersection tests catch both vertex and edge conflicts.

The high-level search follows the priority-based formulation of Ma *et al.* [3]: agents are ordered by a pre-computed Dijkstra distance, the lowest-priority agent replans against the higher-priority fixed paths, and on a conflict the priority ordering is amended. The output is a per-robot `nav_msgs/Path` on `/robot_*/mapf_path`, with each pose carrying a scheduled arrival timestamp.

2) *LNS2 planner*: The LNS2 implementation, `mapf_lns2_node` in `iros_llm_swarm_mapf_lns`, is a C++17 ROS 2 node. The core algorithm follows Li *et al.* [4]: an initial set of single-agent paths is constructed using soft-constraint A\* with collision penalties, and the plan is then iteratively improved by an Adaptive Large-Neighborhood Search (ALNS) loop. At each iteration a destroy operator selects a subset of agents whose paths are removed; four operators are implemented: collision-based, random-walk, random, and bottleneck selection. A repair operator replans those agents using soft-constraint A\*. Operator weights are adapted online based on observed improvement (EMA update per segment).

Three properties are relevant at our scale. First, soft constraints make the algorithm robust to transiently infeasible configurations. Second, LNS2 is anytime: the planner can be interrupted and return the best plan found so far. Third,

replanning is warm-started from the previous plan and from the affected agents’ current poses.

The planner output is an array of `MAPFStep` entries per robot, each carrying a target pose, a scheduled arrival time, and an explicit hold duration (`hold_sec`). The plan also carries a globally unique `plan_id` that the follower echoes back via `FollowerStatus`, allowing the planner to distinguish stale followers (still executing an older plan) from frozen ones (executing the current plan but not making progress).

### D. Long-lived `/swarm/set_goals` action

We model an entire mission as a *single* ROS 2 action goal. The action server runs a state machine

validating → planning → executing → replanning → succeeded

with explicit transitions to failed and cancelled. Schedule monitoring is internal: the server polls per-robot odometry and the latest `FollowerStatus` every period and triggers replanning when any robot deviates beyond a configurable threshold from its scheduled pose or when a follower reports persistent Nav2 failures. Cancellation clears all assigned plans and broadcasts zero velocities.

### E. Per-robot dual-mode followers

Each robot runs one follower node. The PBS stack uses `pbs_motion_controller` (from `iros_llm_swarm_robot`); the LNS2 stack uses `lns_motion_controller` (from `iros_llm_swarm_mapf_lns`). Both maintain a mode variable  $m \in \{\text{IDLE}, \text{AUTONOMOUS}, \text{FORMATION}\}$  and share the same FORMATION-mode behavior.

a) *Autonomous mode (LNS2 follower)*: A new `MAPFPlan` on `/robot_N/mapf_plan` puts the node in **AUTONOMOUS**. The follower walks the plan one step at a time via `Nav2 FollowPath`, honouring hold intervals even if the robot arrives early. The current step index, `plan_id`, hold countdown, and Nav2 failure count are republished on `/robot_N/follower_status`.

b) *Autonomous mode (PBS follower)*: A new `nav_msgs/Path` on `/robot_N/mapf_path` puts the node in **AUTONOMOUS**. The PBS follower walks the path treating each pose’s timestamp as a scheduled arrival time. Unlike the LNS2 follower, this node does *not* publish `FollowerStatus`; the PBS planner detects deviation by polling odometry directly, which is unambiguous because the PBS schedule is deterministic.

c) *Formation mode (both followers)*: A latched message on `/formations/config` that names this robot as a follower puts the node in **FORMATION**. The follower disengages Nav2 and runs a PD controller that tracks the target pose

$$p_{\text{ref}} = p_{\text{leader}} + R(\theta_{\text{leader}}) o,$$

where  $o \in \mathbb{R}^2$  is the body-frame offset declared in the latched configuration. The controller produces twists

$$v = k_p \|e\|, \quad \omega = k_\theta \arctan 2(e_y, e_x) - k_\theta \theta,$$

where  $e$  is the world-frame error vector, subject to limits on linear velocity, angular velocity, and heading rate.

#### F. Formation layer

The formation layer is implemented as two Python nodes: `formation_manager_node` and `formation_monitor_node`.

The manager exposes a CRUD-style service interface: `/formation/set` (create or update), `/formation/activate`, `/formation/deactivate`, `/formation/get`, `/formation/list`, `/formation/remove`, and `/formation/load` / `/formation/save` for YAML persistence. Whenever the registry changes, the manager republishes the full configuration on the latched topic `/formations/config` (transient-local QoS), so a follower joining late receives the current configuration immediately.

The monitor subscribes to the configuration topic and to all relevant odometry topics. For each follower it computes

$$\varepsilon_i = \|p_i - (p_{\text{leader}} + R(\theta_{\text{leader}}) o_i)\|.$$

A state machine classifies the formation as **INACTIVE**, **FORMING**, **STABLE**, **DEGRADED**, or **BROKEN**, with hysteresis on the **STABLE**  $\leftrightarrow$  **DEGRADED** transition. State and per-follower errors are published on `/formations/status` at 10 Hz.

#### G. Behavior-tree integration

We expose the full Level-2 capability set as a small set of `BehaviorTree.CPP` (v3) action nodes:

- `MapfPlan` — async client of `/swarm/set_goals` with cancel-on-halt semantics. On **WARN** or **ERROR** feedback, the node forwards the event to the Level-1 LLM decision server and applies the returned verdict (`wait/abort/replan`).
- `SetFormation/DisableFormation` — thin wrappers over the corresponding `/formation/*` services, with input ports for formation name, leader, follower list, and offsets.
- `CheckMode` — blackboard condition node that compares the latest follower mode against an expected value, useful as a guard in fallback branches.

The reference tree `swarm_navigate_to_pose.xml` sequences these nodes into a complete mission driven by a `ReactiveSequence` that also ticks `BTStatePublisher` and `LlmCommandReceiver` on every cycle.

#### H. ResettingObstacleLayer

Nav2's stock `ObstacleLayer` marks cells along laser returns and clears cells along the rays that reached those returns. In practice, with sparse-angle scans and dynamic obstacles, cells between rays are never explicitly cleared and accumulate as ghost obstacles.

`ResettingObstacleLayer` is a drop-in subclass of `nav2_costmap_2d::ObstacleLayer`. It overrides

`updateBounds()` to call `resetMaps()` on its own internal grid before re-marking from the current scan buffer, guaranteeing that any cell not currently observed as occupied is cleared on every cycle. We pair this with `observation_persistence: 0.0` so no scan is held over from the previous cycle.

#### I. Engineering decisions at scale

a) *CycloneDDS profile*: The default RMW configuration saturates discovery traffic above roughly ten participants. We ship `cyclonedds_swarm.xml`, which restricts unicast discovery, tunes participant lease durations, and fixes the network interface. Without this profile the system fails to fully discover all 20 robot namespaces within the launch budget.

b) *Static localization*: AMCL is disabled. With 20 robots in a shared 2D environment, each robot's laser scan contains the bodies of every other robot, which the particle filter treats as unexplained dynamic obstacles; filter weights collapse. We install a static map  $\rightarrow$  `robot_N/odom` identity transform per robot and rely on simulator odometry as ground truth.

c) *Staggered launch*: The bring-up applies a coarse schedule (simulator at 0 s, Nav2 at 1 s, planner at 10 s, followers at 12 s) to allow DDS discovery and costmap initialization to settle. Per-robot Nav2 instances are also staggered by  $\sim 0.3$  s within the Nav2 step.

### IV. CODE AVAILABILITY

The full implementation is hosted at:

[https://github.com/innopolis-robotics-society/llm\\_swarm](https://github.com/innopolis-robotics-society/llm_swarm)

The repository contains a top-level `README.md` with Docker bring-up instructions, scenario launch commands, and a description of the `/swarm/set_goals` JSON goal format. Per-package documentation is generated by `rostdoc2` and aggregated into a Sphinx site under `iros_llm_swarm_docs/`.

### V. EVALUATION

All tests were run inside the project's Docker image on a single workstation with `ROS_DOMAIN_ID=42`, `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp`, and the project CycloneDDS profile. The simulator is Stage 2D (`stage_ros2`) with the project robot model.

#### A. MAPF solver correctness

The LNS2 solver is validated by two compiled standalone tests. A *smoke test* places three agents on crossing straight paths in a  $20 \times 20$  empty grid (0.2 m/cell) and calls `LNS2Solver::solve()` with a 1 s budget; the test passes when `ok=true` and `final_collisions=0`. A *corridor stress test* places eight agents in four opposing pairs that must share a four-cell-wide corridor in a  $30 \times 30$  grid with a blocked interior, with a 3 s budget; the test passes when all head-on conflicts are resolved. Both tests use fixed seeds and pass deterministically in CI.

These tests also exercise the four ALNS destroy operators (collision-based, random-walk, random, bottleneck) and verify

that adaptive weight updates converge: the stress test reports per-operator usage and improvement counts that confirm all four operators are selected and that their weights shift toward the operators that produce the most improvement on the corridor instance.

### B. End-to-end integration scenario

The `test_bt_runner` node exercises the full Level-2 stack against a live 20-robot Stage simulation with the LNS2 backend, without a human or LLM operator. A scripted six-step scenario runs in the warehouse world (30×30 m):

- 1) All 20 robots converge to the warehouse centre at (15, 15) in a 4×5 grid with 1.5 m spacing.
- 2) A WEDGE formation is activated over the orange squad (robots 0–9, robot 1 as leader).
- 3) A LINE formation is activated over the blue squad (robots 10–14, robot 10 as leader).
- 4) A full cross-swap sends orange robots to the top-right region and blue robots to the bottom-left, stressing MAPF conflict resolution over the 30 m diagonal with active formations.
- 5) All 20 robots return to their original home positions.
- 6) Idle.

The scenario completes successfully. Steps 1, 4, and 5 exercise MAPF planning and schedule tracking; steps 2 and 3 exercise the formation manager’s CRUD interface and the PD controller; step 4 is the primary stress test, as two fully formed squads must cross without collision.

### C. Formation service interface

The formation manager’s full service interface—`/formation/set`, `activate`, `deactivate`, `get`, `list`, `remove`, `load`, `save`—was exercised via the dedicated BT test trees (`test_set_formation.xml`, `test_disable_formation.xml`) and the `test_send_goals.py` script, which supports goal-file JSON input for reproducible formation scenarios. Formation validation (checking that a new formation configuration is geometrically consistent before activation) was added following integration testing that revealed silent acceptance of malformed offset arrays.

### D. Limitations

Several aspects have not been subjected to systematic quantitative evaluation and are open items for future work.

*Backend comparison.* A controlled sweep of PBS vs. LNS2 across fleet sizes, scenario types, and goal distributions—measuring first-plan time, sum-of-costs, makespan, and replans per mission—has not been carried out. The qualitative regime distinction (PBS for sparse/sparse, LNS2 for dense/many-replans) is based on development experience, not on a measurement campaign.

*Schedule adherence under execution.* Per-robot schedule deviation  $\Delta_i(t)$  and the replan rate as a function of Nav2 controller failure rate have not been measured systematically.

The integration scenario passes, but no statistics over repeated runs have been collected.

*Formation tracking accuracy.* The PD controller’s offset-tracking error as a function of leader velocity and formation shape (line, wedge, triangle, square) has not been characterized. The formation is observed to be stable in Step 4 of the integration scenario, but no quantitative  $\varepsilon_i$  statistics are available.

*ResettingObstacleLayer ablation.* A direct comparison of mission success rate and recovery behavior invocations between the stock `ObstacleLayer` and our plugin under the 20-robot scan density has not been run as a controlled experiment.

*DDS profile quantification.* The bring-up failure rate without the CycloneDDS profile has been observed qualitatively but not measured as a fraction over repeated cold-boots.

## VI. ANALYSIS AND OBSERVATIONS

*a) Each planner has its regime:* Keeping both backends behind a single action paid off the moment we tried to run the same scenarios under both. PBS is cheap, deterministic, and easy to reason about on sparse instances; we use it for reproducible regression runs and when the schedule is treated as a verifiable contract. The cost shows up when start poses crowd: PBS’s hard, capsule-style separation declares a non-trivial fraction of instances infeasible during pre-planning. The same instances are recoverable under LNS2 because soft-constraint A\* makes progress on penalised paths and the ALNS loop resolves residual conflicts. The operational rule we converged on: PBS by default, LNS2 whenever start density or scenario complexity makes infeasibility likely or when many replans are expected.

*b) Long-lived actions simplify the upstream layer:* Once we exposed an entire mission as a single action goal, the behavior tree shrank dramatically. The original tree had separate nodes for “request plan”, “wait for plan”, “dispatch paths”, and “monitor execution”. Folding the state machine into the server replaces all of these with a single `MapfPlan` node, which makes the tree readable and makes it trivial to add cancellation as a tree-level fallback.

*c) Mode switching must be event-driven:* An earlier prototype restarted the follower node on formation/autonomous transitions. Restarts caused a 1–2 s control gap during which the robot drifted and the costmap re-initialized, frequently inducing a downstream Nav2 failure. Treating the mode as a state variable updated by latched messages eliminates the gap entirely.

*d) AMCL is not workable at this density:* We attempted several AMCL configurations before disabling it. Tightening the laser likelihood field made the filter overconfident on spurious matches against neighbouring robot bodies; widening it caused the filter to refuse to converge during initial spawning. The static identity transform is not a long-term solution—hardware deployment will require a multi-robot SLAM pipeline—but it is correct and stable in simulation.

e) *Costmap clearing is a real failure mode:* Without `ResettingObstacleLayer`, robots leave a trail of lethal cells across the costmap, which causes the local planner either to abort with no valid trajectory or to invoke a recovery behavior that desynchronizes the fleet from the schedule. The plugin replaces this failure mode with a small, bounded computational cost per costmap cycle.

f) *DDS dominates the bring-up budget:* On the wall-clock budget of a 20-robot launch, the largest single cost is DDS discovery, not planning or simulation. Without the project profile, a non-trivial fraction of bring-up runs fails to register all robots within the staggered timeline, and the planner action server then computes a plan over a partial fleet.

## VII. CONCLUSION AND FUTURE WORK

We described the Level-2 layer of a hierarchical multi-robot control stack: two interchangeable MAPF backends—PBS with 8-connected A\* and capsule-based conflict detection, and LNS2 with ALNS over a soft-constraint A\*—sharing a single fleet-level action surface; a long-lived mission action with internal schedule monitoring; a centralized formation manager with a state-machine monitor; per-robot dual-mode followers that unify Nav2 path-following and PD offset tracking; a self-clearing costmap layer; and a small set of behavior-tree nodes. The system runs at the target fleet size of 20 robots and the six-step integration scenario passes end to end.

Future work falls into four directions. First, *formation-aware MAPF*: formations and MAPF are currently mutually exclusive operational modes; adding formation-rigidity terms to both planner cost models would allow follower paths to be planned jointly with the leader. Second, *fault tolerance under communication loss*: the `FollowerStatus` channel already distinguishes frozen from holding followers; missing-status policies and graceful degradation under partial DDS failure are the next step. Third, *anytime budget tuning*: the LNS2 iteration count and PBS priority-resolution depth are currently fixed; an online controller that trades quality for latency under a budget is a natural extension. Fourth, *quantitative evaluation*: the comparisons described in Section V as limitations constitute the primary open measurement task.

## REFERENCES

- [1] R. Stern *et al.*, “Multi-agent pathfinding: Definitions, variants, and benchmarks,” in *Proc. SoCS*, 2019.
- [2] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artif. Intell.*, vol. 219, pp. 40–66, 2015.
- [3] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig, “Searching with consistent prioritization for multi-agent path finding,” in *Proc. AAAI*, 2019.
- [4] J. Li, Z. Chen, D. Harabor, P. J. Stuckey, and S. Koenig, “Anytime multi-agent path finding via large neighborhood search,” in *Proc. IJCAI*, 2021.
- [5] V. G. Santos *et al.*, “Spatial segregative behaviors in robotic swarms using differential potentials,” *Swarm Intelligence*, vol. 14, no. 4, pp. 259–284, 2020.
- [6] H. Wang, M. Yue, X. Sun, and X. Zhao, “Cooperative formation control strategy for multi-robot system based on APF algorithm and sliding mode estimator,” *Robotics and Autonomous Systems*, vol. 193, p. 105075, 2025.
- [7] C. Jiang, Z. Chen, and Y. Guo, “Multi-robot formation control: a comparison between model-based and learning-based methods,” *J. Control Decis.*, vol. 7, no. 1, pp. 90–108, 2020.
- [8] F. Martinez and A. Rendon, “Autonomous motion planning for a differential robot using particle swarm optimization,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 14, 2023.
- [9] E. Debie, K. Kasmarik, and M. Garratt, “Swarm robotics: A survey from a multi-tasking perspective,” *ACM Comput. Surv.*, vol. 56, no. 2, 2023.
- [10] K. Elamvazhuthi and S. Berman, “Mean-field models in swarm robotics: A survey,” 2019.
- [11] A. M. Elsayed, M. Elshalakani, S. A. Hammad, and S. A. Maged, “Decentralized fault-tolerant control of multi-mobile robot system addressing LiDAR sensor faults,” *Sci. Rep.*, vol. 14, no. 1, p. 25713, 2024.
- [12] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, “The Marathon 2: A navigation system,” in *Proc. IROS*, 2020.
- [13] S. Macenski *et al.*, “From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2,” *Robot. Auton. Syst.*, 2023.
- [14] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, 2022.
- [15] D. V. Lu, D. Hershberger, and W. D. Smart, “Layered costmaps for context-sensitive navigation,” in *Proc. IROS*, 2014.
- [16] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.
- [17] M. Iovino, E. Scukins, J. Styrd, P. Ögren, and C. Smith, “A survey of behavior trees in robotics and AI,” *Robot. Auton. Syst.*, vol. 154, 2022.
- [18] D. Faconti and M. Colledanchise, “BehaviorTree.CPP,” <https://www.behaviortree.dev>, accessed 2026.
- [19] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ROS2,” in *Proc. EMSOFT*, 2016.
- [20] T. Kronauer, J. Pohlmann, M. Matthe, T. Smejkal, and G. Fettweis, “Latency analysis of ROS2 multi-node systems,” 2023.
- [21] R. Vaughan, “Massively multi-robot simulation in Stage,” *Swarm Intell.*, vol. 2, pp. 189–208, 2008.
- [22] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Proc. IROS*, 2004.
- [23] M. Chamanbaz *et al.*, “Swarm-enabling technology for multi-robot systems,” *Front. Robot. AI*, vol. 4, p. 12, 2017.
- [24] P. Kedia and M. Rao, “Gengrid: A generalised distributed experimental environmental grid for swarm robotics,” arXiv:2504.20071, 2025.
- [25] M. Ahn *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proc. CoRL*, 2022.
- [26] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” in *Proc. ICRA*, 2023.
- [27] W. Huang *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Proc. CoRL*, 2022.
- [28] I. Singh *et al.*, “ProgPrompt: Generating situated robot task plans using large language models,” in *Proc. ICRA*, 2023.